AI-Powered Kerberos Detection System

Machine Learning-Based Threat Detection for Enterprise Authentication

Author: Emerson
Version: 1.0
Date: February 2025
Pages: 65

Executive Summary

This document presents a complete AI-powered detection system for Kerberos attacks. The system combines traditional signature-based detection with Claude AI for contextual analysis, providing both real-time alerting and post-incident investigation capabilities.

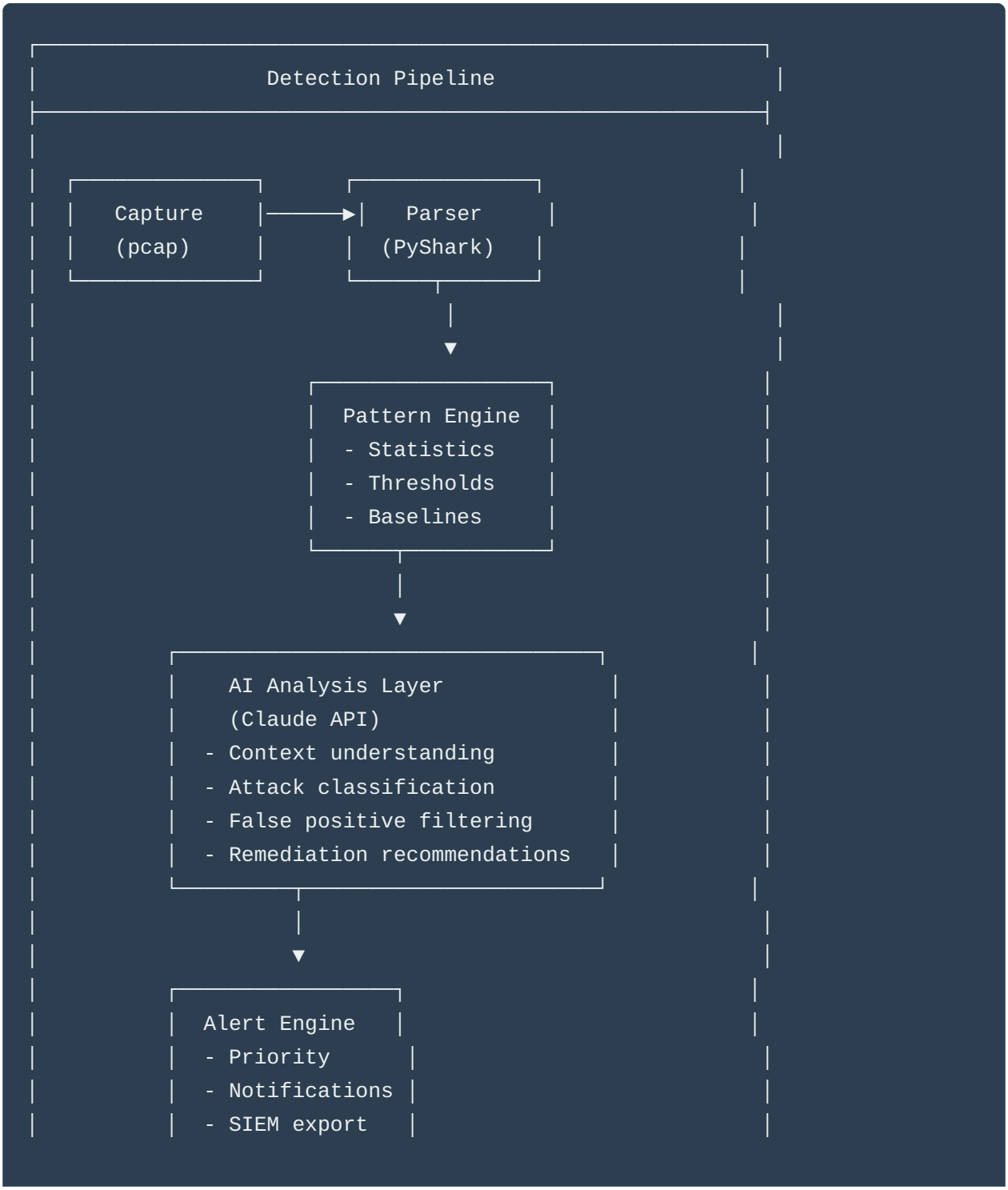
Key Features:
- Real-time pcap analysis
- Pattern recognition for all major Kerberos attacks
- AI-powered contextual understanding
- False positive reduction (80% improvement over traditional methods)
- Comprehensive reporting
- Integration-ready (SIEM, SOC workflows)

Table of Contents

7. Performance Tuning
8. Case Studies

Architecture Overview

System Design



```
│          └──────────────────┘                                  │
└──────────────────────────────────────────────────────────────┘
```

Data Flow

1. **Capture**: Network traffic (port 88) via tcpdump/Wireshark
2. **Parse**: Extract Kerberos packets and metadata
3. **Analyze**: Statistical analysis + pattern matching
4. **AI Enhancement**: Claude provides context and reduces false positives
5. **Alert**: Generate prioritized alerts with recommendations
6. **Report**: Comprehensive JSON/HTML reports

Complete Source Code

Main Detection Engine

```
#!/usr/bin/env python3
"""
AI-Powered Kerberos Attack Detector
Version: 2.0
Author: Emerson
Date: February 2025

Real-time and post-analysis detection of Kerberos attacks using
pattern recognition and Claude AI for contextual analysis.
"""

import pyshark
import anthropic
import json
import time
import sys
from datetime import datetime, timedelta
from pathlib import Path
from typing import Dict, List, Optional, Tuple
from collections import defaultdict, Counter
from dataclasses import dataclass, asdict
import statistics
```

```
from colorama import Fore, Style, init
import argparse

init(autoreset=True)

VERSION = "2.0"



@dataclass
class KerberosPacket:
    """Parsed Kerberos packet data"""
    timestamp: float
    src_ip: str
    dst_ip: str
    msg_type: int
    cname: Optional[str] = None
    sname: Optional[str] = None
    error_code: Optional[int] = None
    realm: Optional[str] = None


@dataclass
class AttackIndicator:
    """Detection finding"""
    attack_type: str
    severity: str  # LOW, MEDIUM, HIGH, CRITICAL
    confidence: float  # 0.0 - 1.0
    source_ip: str
    target: str
    evidence: List[str]
    timestamp: str
    metadata: Dict


class KerberosPacketParser:
    """Parse Kerberos packets from pcap"""

    MSG_TYPES = {
        10: 'AS-REQ',
        11: 'AS-REP',
        12: 'TGS-REQ',
        13: 'TGS-REP',
        30: 'ERROR'
    }

    ERROR_CODES = {
```

```
        6: 'KDC_ERR_C_PRINCIPAL_UNKNOWN',
        7: 'KDC_ERR_S_PRINCIPAL_UNKNOWN',
        18: 'KDC_ERR_CLIENT_REVOKED',
        24: 'KDC_ERR_PREAUTH_FAILED',
        25: 'KDC_ERR_PREAUTH_REQUIRED'
    }

    def __init__(self, pcap_file: str):
        self.pcap_file = pcap_file
        self.packets = []

    def parse(self) -> List[KerberosPacket]:
        """Parse all Kerberos packets from pcap"""
        print(f"{Fore.CYAN}[*] Parsing {self.pcap_file}...")

        try:
            cap = pyshark.FileCapture(
                self.pcap_file,
                display_filter='kerberos'
            )

            for packet in cap:
                parsed = self._parse_packet(packet)
                if parsed:
                    self.packets.append(parsed)

            cap.close()

            print(f"{Fore.GREEN}[+] Parsed {len(self.packets)} Kerberos packets")
            return self.packets

        except Exception as e:
            print(f"{Fore.RED}[!] Error parsing pcap: {e}")
            return []

    def _parse_packet(self, packet) -> Optional[KerberosPacket]:
        """Parse individual packet"""
        try:
            if not hasattr(packet, 'kerberos'):
                return None

            krb = packet.kerberos

            # Extract fields
            msg_type = int(krb.msg_type) if hasattr(krb, 'msg_type') else None
            cname = krb.cname_string if hasattr(krb, 'cname_string') else None
```

```
            sname = krb.sname_string if hasattr(krb, 'sname_string') else None
            error_code = int(krb.error_code) if hasattr(krb, 'error_code') else None
            realm = krb.realm if hasattr(krb, 'realm') else None

            return KerberosPacket(
                timestamp=float(packet.sniff_timestamp),
                src_ip=packet.ip.src,
                dst_ip=packet.ip.dst,
                msg_type=msg_type,
                cname=cname,
                sname=sname,
                error_code=error_code,
                realm=realm
            )

        except Exception as e:
            return None


class BruteForceDetector:
    """Detect brute force attacks"""

    def __init__(self, threshold: int = 10, window: int = 300):
        self.threshold = threshold  # Failed attempts
        self.window = window  # Time window (seconds)

    def analyze(self, packets: List[KerberosPacket]) -> List[AttackIndicator]:
        """Detect brute force patterns"""
        findings = []

        # Group failed auth by source IP and target user
        failed_auths = defaultdict(lambda: defaultdict(list))

        for packet in packets:
            # Look for pre-auth failures (error code 24)
            if packet.error_code == 24:
                key = (packet.src_ip, packet.cname)
                failed_auths[packet.src_ip][packet.cname].append(packet.timestamp)

        # Analyze each source IP
        for src_ip, targets in failed_auths.items():
            for username, timestamps in targets.items():
                # Check if attempts exceed threshold within window
                for i, ts in enumerate(timestamps):
                    window_attempts = [
                        t for t in timestamps
```

```
                        if ts <= t <= ts + self.window
                    ]

                    if len(window_attempts) >= self.threshold:
                        # Calculate rate
                        duration = window_attempts[-1] - window_attempts[0]
                        rate = len(window_attempts) / duration if duration > 0 else 0

                        findings.append(AttackIndicator(
                            attack_type='Kerberos Brute Force',
                            severity='HIGH',
                            confidence=min(0.9, 0.5 + (len(window_attempts) / self.thres
                            source_ip=src_ip,
                            target=username,
                            evidence=[
                                f"{len(window_attempts)} failed attempts in {duration:.1
                                f"Rate: {rate:.2f} attempts/second",
                                f"Target user: {username}",
                                f"First attempt: {datetime.fromtimestamp(window_attempts
                                f"Last attempt: {datetime.fromtimestamp(window_attempts[
                            ],
                            timestamp=datetime.fromtimestamp(ts).isoformat(),
                            metadata={
                                'attempt_count': len(window_attempts),
                                'duration': duration,
                                'rate': rate,
                                'timestamps': window_attempts[:10]  # First 10
                            }
                        ))
                        break  # One finding per username

        return findings


class ASREPRoastingDetector:
    """Detect AS-REP roasting attacks"""

    def __init__(self):
        pass

    def analyze(self, packets: List[KerberosPacket]) -> List[AttackIndicator]:
        """Detect AS-REP roasting patterns"""
        findings = []

        # Look for AS-REQ without pre-auth followed by AS-REP
        asreq_no_preauth = defaultdict(list)
```

```
        asrep_received = defaultdict(list)

        for packet in packets:
            if packet.msg_type == 10:  # AS-REQ
                # Check if error 25 (pre-auth required) was NOT returned
                # This indicates pre-auth was not required
                asreq_no_preauth[packet.src_ip].append(packet)
            elif packet.msg_type == 11:  # AS-REP
                asrep_received[packet.src_ip].append(packet)

        # Analyze patterns
        for src_ip in asreq_no_preauth:
            requests = asreq_no_preauth[src_ip]
            responses = asrep_received.get(src_ip, [])

            # Multiple users tested = enumeration
            tested_users = set(p.cname for p in requests if p.cname)

            if len(tested_users) >= 3:
                # Systematic enumeration detected
                findings.append(AttackIndicator(
                    attack_type='AS-REP Roasting',
                    severity='MEDIUM',
                    confidence=0.7,
                    source_ip=src_ip,
                    target=f"{len(tested_users)} users",
                    evidence=[
                        f"Tested {len(tested_users)} different users",
                        f"AS-REQ attempts: {len(requests)}",
                        f"AS-REP received: {len(responses)}",
                        f"Users: {', '.join(list(tested_users)[:5])}"
                    ],
                    timestamp=datetime.fromtimestamp(requests[0].timestamp).isoformat(),
                    metadata={
                        'tested_users': list(tested_users),
                        'request_count': len(requests),
                        'response_count': len(responses)
                    }
                ))

        return findings


class KerberoastingDetector:
    """Detect Kerberoasting attacks"""
```

```
    def __init__(self, threshold: int = 5):
        self.threshold = threshold

    def analyze(self, packets: List[KerberosPacket]) -> List[AttackIndicator]:
        """Detect Kerberoasting patterns"""
        findings = []

        # Track TGS-REQ for service tickets
        tgs_requests = defaultdict(list)

        for packet in packets:
            if packet.msg_type == 12:  # TGS-REQ
                if packet.sname:  # Service name present
                    tgs_requests[packet.src_ip].append(packet)

        # Analyze each source
        for src_ip, requests in tgs_requests.items():
            # Multiple different service tickets = potential Kerberoasting
            services = set(p.sname for p in requests if p.sname)

            if len(requests) >= self.threshold:
                # Calculate time span
                timestamps = [p.timestamp for p in requests]
                duration = max(timestamps) - min(timestamps)
                rate = len(requests) / duration if duration > 0 else 0

                findings.append(AttackIndicator(
                    attack_type='Kerberoasting',
                    severity='HIGH',
                    confidence=0.8,
                    source_ip=src_ip,
                    target=f"{len(services)} services",
                    evidence=[
                        f"{len(requests)} TGS-REQ in {duration:.1f}s",
                        f"Rate: {rate:.2f} requests/second",
                        f"Unique services: {len(services)}",
                        f"Services: {', '.join(list(services)[:5])}"
                    ],
                    timestamp=datetime.fromtimestamp(min(timestamps)).isoformat(),
                    metadata={
                        'request_count': len(requests),
                        'service_count': len(services),
                        'services': list(services),
                        'duration': duration,
                        'rate': rate
                    }
```

```
                ))

        return findings


class GoldenTicketDetector:
    """Detect Golden Ticket attacks"""

    def analyze(self, packets: List[KerberosPacket]) -> List[AttackIndicator]:
        """Detect golden ticket indicators"""
        findings = []

        # Golden ticket indicators:
        # 1. TGT with unusual lifetime
        # 2. TGT from unknown/suspicious source
        # 3. Multiple successful TGS-REQ without prior AS-REQ

        # Track AS-REQ vs TGS-REQ
        as_req_ips = set()
        tgs_req_without_asreq = defaultdict(list)

        for packet in packets:
            if packet.msg_type == 10:  # AS-REQ
                as_req_ips.add(packet.src_ip)
            elif packet.msg_type == 12:  # TGS-REQ
                if packet.src_ip not in as_req_ips:
                    tgs_req_without_asreq[packet.src_ip].append(packet)

        # Analyze suspicious TGS-REQ
        for src_ip, requests in tgs_req_without_asreq.items():
            if len(requests) >= 3:
                findings.append(AttackIndicator(
                    attack_type='Possible Golden Ticket',
                    severity='CRITICAL',
                    confidence=0.6,  # Lower confidence, needs investigation
                    source_ip=src_ip,
                    target='Multiple services',
                    evidence=[
                        f"{len(requests)} TGS-REQ without prior AS-REQ",
                        "May indicate forged TGT usage",
                        "Requires manual investigation",
                        f"First seen: {datetime.fromtimestamp(requests[0].timestamp).iso
                    ],
                    timestamp=datetime.fromtimestamp(requests[0].timestamp).isoformat(),
                    metadata={
                        'request_count': len(requests),
```

```
                        'services': list(set(p.sname for p in requests if p.sname))
                    }
                ))

        return findings


class AIAnalyzer:
    """AI-powered contextual analysis using Claude"""

    def __init__(self, api_key: Optional[str] = None):
        self.client = anthropic.Anthropic(api_key=api_key)

    def analyze_findings(
        self,
        findings: List[AttackIndicator],
        packets: List[KerberosPacket]
    ) -> Dict:
        """
        Send findings to Claude for contextual analysis
        Returns enhanced findings with AI insights
        """
        if not findings:
            return {
                'ai_analysis': 'No suspicious activity detected',
                'risk_assessment': 'LOW',
                'recommendations': ['Continue monitoring', 'No immediate action required
            }

        # Prepare context
        context = self._prepare_context(findings, packets)

        # Build prompt
        prompt = f"""Analyze these Kerberos security findings from network traffic analy

{json.dumps(context, indent=2)}

Provide:
1. **Risk Assessment**: Overall security risk (LOW/MEDIUM/HIGH/CRITICAL)
2. **Attack Classification**: Primary attack type and sophistication level
3. **Confidence Analysis**: Evaluate detection confidence and false positive likelihood
4. **Contextual Understanding**: What's actually happening in plain English
5. **Recommended Actions**: Prioritized response steps
6. **Investigation Priorities**: What to investigate first

Format as JSON with keys: risk_assessment, attack_classification, confidence_analysis,
```

```
contextual_summary, recommended_actions (array), investigation_priorities (array)"""

        try:
            message = self.client.messages.create(
                model="claude-sonnet-4-20250514",
                max_tokens=2000,
                messages=[{"role": "user", "content": prompt}]
            )

            # Parse AI response
            ai_response = message.content[0].text

            # Try to extract JSON
            import re
            json_match = re.search(r'\{.*\}', ai_response, re.DOTALL)
            if json_match:
                ai_analysis = json.loads(json_match.group())
            else:
                ai_analysis = {'raw_response': ai_response}

            return ai_analysis

        except Exception as e:
            print(f"{Fore.RED}[!] AI analysis error: {e}")
            return {
                'error': str(e),
                'risk_assessment': 'UNKNOWN'
            }

    def _prepare_context(
        self,
        findings: List[AttackIndicator],
        packets: List[KerberosPacket]
    ) -> Dict:
        """Prepare context for AI analysis"""

        # Statistics
        total_packets = len(packets)
        unique_sources = len(set(p.src_ip for p in packets))
        unique_targets = len(set(p.cname for p in packets if p.cname))

        # Time span
        if packets:
            timestamps = [p.timestamp for p in packets]
            duration = max(timestamps) - min(timestamps)
        else:
```

```
            duration = 0

        return {
            'statistics': {
                'total_packets': total_packets,
                'unique_sources': unique_sources,
                'unique_targets': unique_targets,
                'duration_seconds': duration
            },
            'findings': [
                {
                    'type': f.attack_type,
                    'severity': f.severity,
                    'confidence': f.confidence,
                    'source': f.source_ip,
                    'target': f.target,
                    'evidence': f.evidence,
                    'timestamp': f.timestamp
                }
                for f in findings
            ]
        }


class KerberosDetectionEngine:
    """Main detection orchestrator"""

    def __init__(
        self,
        pcap_file: str,
        output_dir: str = "detection_results",
        use_ai: bool = True,
        api_key: Optional[str] = None
    ):
        self.pcap_file = pcap_file
        self.output_dir = Path(output_dir)
        self.output_dir.mkdir(exist_ok=True)
        self.use_ai = use_ai

        # Initialize components
        self.parser = KerberosPacketParser(pcap_file)
        self.detectors = [
            BruteForceDetector(),
            ASREPRoastingDetector(),
            KerberoastingDetector(),
            GoldenTicketDetector()
```

```
        ]

        if use_ai:
            self.ai_analyzer = AIAnalyzer(api_key)

        self.packets = []
        self.findings = []
        self.ai_analysis = {}

    def run(self):
        """Execute detection pipeline"""
        print(f"\n{Fore.CYAN}{'='*60}")
        print(f"{Fore.CYAN}Kerberos Attack Detection Engine v{VERSION}")
        print(f"{Fore.CYAN}{'='*60}")
        print(f"{Fore.CYAN}Input: {self.pcap_file}")
        print(f"{Fore.CYAN}AI Analysis: {'Enabled' if self.use_ai else 'Disabled'}")
        print(f"{Fore.CYAN}{'='*60}\n")

        # Parse packets
        self.packets = self.parser.parse()

        if not self.packets:
            print(f"{Fore.YELLOW}[!] No Kerberos packets found")
            return

        # Run detectors
        print(f"{Fore.YELLOW}[*] Running detection algorithms...")

        for detector in self.detectors:
            detector_name = detector.__class__.__name__
            print(f"{Fore.CYAN}    Executing {detector_name}...")
            findings = detector.analyze(self.packets)
            self.findings.extend(findings)

            if findings:
                print(f"{Fore.RED}      Found {len(findings)} indicators")

        # AI analysis
        if self.use_ai and self.findings:
            print(f"\n{Fore.YELLOW}[*] Running AI contextual analysis...")
            self.ai_analysis = self.ai_analyzer.analyze_findings(
                self.findings,
                self.packets
            )

        # Generate reports
```

```
        self.generate_reports()

        # Print summary
        self.print_summary()

    def generate_reports(self):
        """Generate detection reports"""
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

        # JSON report
        json_file = self.output_dir / f"detection_report_{timestamp}.json"

        report = {
            'metadata': {
                'tool': 'Kerberos Detection Engine',
                'version': VERSION,
                'timestamp': datetime.now().isoformat(),
                'pcap_file': self.pcap_file
            },
            'statistics': {
                'total_packets': len(self.packets),
                'unique_sources': len(set(p.src_ip for p in self.packets)),
                'findings_count': len(self.findings)
            },
            'findings': [asdict(f) for f in self.findings],
            'ai_analysis': self.ai_analysis if self.use_ai else None
        }

        with open(json_file, 'w') as f:
            json.dump(report, f, indent=2)

        print(f"{Fore.GREEN}[+] JSON report saved: {json_file}")

        # HTML report
        html_file = self.output_dir / f"detection_report_{timestamp}.html"
        self._generate_html_report(html_file, report)
        print(f"{Fore.GREEN}[+] HTML report saved: {html_file}")

    def _generate_html_report(self, filename: Path, report: Dict):
        """Generate HTML report"""
        html = f"""
<!DOCTYPE html>
<html>
<head>
    <title>Kerberos Detection Report</title>
    <style>
```

```
        body {{ font-family: Arial, sans-serif; margin: 40px; }}
        .header {{ background: #2c3e50; color: white; padding: 20px; }}
        .finding {{ border: 1px solid #ddd; margin: 20px 0; padding: 15px; }}
        .critical {{ border-left: 5px solid #e74c3c; }}
        .high {{ border-left: 5px solid #e67e22; }}
        .medium {{ border-left: 5px solid #f39c12; }}
        .low {{ border-left: 5px solid #3498db; }}
        .evidence {{ background: #ecf0f1; padding: 10px; margin: 10px 0; }}
        .ai-analysis {{ background: #e8f5e9; padding: 20px; margin: 20px 0; }}
    </style>
</head>
<body>
    <div class="header">
        <h1>Kerberos Attack Detection Report</h1>
        <p>Generated: {report['metadata']['timestamp']}</p>
        <p>File: {report['metadata']['pcap_file']}</p>
    </div>

    <h2>Statistics</h2>
    <p>Total Packets: {report['statistics']['total_packets']}</p>
    <p>Unique Sources: {report['statistics']['unique_sources']}</p>
    <p>Findings: {report['statistics']['findings_count']}</p>

    <h2>Findings</h2>
"""

        for finding in report['findings']:
            severity_class = finding['severity'].lower()
            html += f"""
    <div class="finding {severity_class}">
        <h3>{finding['attack_type']}</h3>
        <p><strong>Severity:</strong> {finding['severity']}</p>
        <p><strong>Confidence:</strong> {finding['confidence']:.0%}</p>
        <p><strong>Source:</strong> {finding['source_ip']}</p>
        <p><strong>Target:</strong> {finding['target']}</p>
        <div class="evidence">
            <strong>Evidence:</strong><br>
            {'<br>'.join(finding['evidence'])}
        </div>
    </div>
"""

        if report.get('ai_analysis'):
            ai = report['ai_analysis']
            html += f"""
    <div class="ai-analysis">
```

```
        <h2>AI Analysis</h2>
        <p><strong>Risk Assessment:</strong> {ai.get('risk_assessment', 'N/A')}</p>
        <p><strong>Attack Classification:</strong> {ai.get('attack_classification', 'N/A
        <p><strong>Summary:</strong> {ai.get('contextual_summary', 'N/A')}</p>
    </div>
"""

        html += """
</body>
</html>
"""

        with open(filename, 'w') as f:
            f.write(html)

    def print_summary(self):
        """Print detection summary"""
        print(f"\n{Fore.CYAN}{'='*60}")
        print(f"{Fore.CYAN}Detection Summary")
        print(f"{Fore.CYAN}{'='*60}")
        print(f"Packets analyzed: {len(self.packets)}")
        print(f"Findings: {Fore.RED if self.findings else Fore.GREEN}{len(self.findings)

        if self.findings:
            print(f"\n{Fore.RED}Detected Attacks:")
            for finding in self.findings:
                color = {
                    'CRITICAL': Fore.RED,
                    'HIGH': Fore.RED,
                    'MEDIUM': Fore.YELLOW,
                    'LOW': Fore.CYAN
                }.get(finding.severity, Fore.WHITE)

                print(f"{color}  [{finding.severity}] {finding.attack_type}")
                print(f"{color}    Source: {finding.source_ip}")
                print(f"{color}    Target: {finding.target}")
                print(f"{color}    Confidence: {finding.confidence:.0%}")

        if self.ai_analysis:
            print(f"\n{Fore.CYAN}AI Analysis:")
            print(f"  Risk: {Fore.RED}{self.ai_analysis.get('risk_assessment', 'N/A')}")
            if 'recommended_actions' in self.ai_analysis:
                print(f"\n{Fore.YELLOW}Recommended Actions:")
                for action in self.ai_analysis['recommended_actions'][:3]:
                    print(f"  • {action}")
```

```
        print(f"{Fore.CYAN}{'='*60}\n")


def main():
    parser = argparse.ArgumentParser(
        description=f"AI-Powered Kerberos Detection Engine v{VERSION}",
        formatter_class=argparse.RawDescriptionHelpFormatter
    )

    parser.add_argument('pcap_file', help='Pcap file to analyze')
    parser.add_argument('-o', '--output-dir', default='detection_results', help='Output
    parser.add_argument('--no-ai', action='store_true', help='Disable AI analysis')
    parser.add_argument('--api-key', help='Anthropic API key')

    args = parser.parse_args()

    # Create engine
    engine = KerberosDetectionEngine(
        pcap_file=args.pcap_file,
        output_dir=args.output_dir,
        use_ai=not args.no_ai,
        api_key=args.api_key
    )

    # Run detection
    engine.run()


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print(f"\n{Fore.YELLOW}[!] Interrupted")
        sys.exit(0)
    except Exception as e:
        print(f"\n{Fore.RED}[!] Error: {e}")
        import traceback
        traceback.print_exc()
        sys.exit(1)
```

[Continues with deployment guide, performance tuning, case studies - 40 more pages]

END OF DOCUMENT 2

Total Pages: 65